

Week 3 · 개념 워크북: state를 내 노트북에서 원격 저장소로 옮깁니다

참고

이 문서는 개념 파트입니다. 직접 입력하는 실습은 같은 `lecture/` 폴더의 `실습워크북` 을 따라가세요. `[개념]` 은 개념 설명, `[함정]` 은 미리 알고 피해야 할 실수입니다.

지난주에 `terraform.tfstate` 를 열어봤습니다. 리소스 7개가 어떤 값으로 만들어졌는지, 그리고 `dependencies` 에 삭제 순서까지 적혀 있는 것을 봤습니다. 그 파일은 지금 여러분 노트북 한 곳에만 있습니다. 오늘은 그 파일을 S3로 옮기고, 두 사람이 동시에 수정하지 못하게 잠금을 설정합니다.

이 문서를 마쳤을 때 세 가지를 말할 수 있어야 합니다. 왜 스택을 둘로 나누는지 닭-달걀 문제로 설명하기, `init -migrate-state` 가 무엇을 읽고 무엇을 쓰는지 순서로 말하기, 잠금이 항목 하나로 어떻게 동작하는지 설명하기.

체크포인트

week1·2에서 이미 설명한 것은 다시 설명하지 않고 위치만 가리킵니다. VPC · 서브넷 · IGW · 라우트 테이블 · 시큐리티 그룹 · AMI · CIDR · 참조와 그래프는 week2 개념워크북 Part 0 · Part 1 · Part 2에 있습니다.

오늘은 week2 개념워크북 14번이 Week3로 넘긴 표의 내용을 오늘 다룹니다.

week2에서 넘긴 것	오늘 어디서 다루는가
<code>terraform.tfstate</code> 가 내 노트북에만 있다. 팀원은 이 state를 볼 수 없다	Part 1 · Part 2
<code>serial</code> 이 저장할 때마다 1씩 증가한다. 두 사람이 동시에 apply하면 충돌한다	Part 3
state는 평문 JSON이다. 원격에 두려면 암호화와 접근 제어가 필요하다	Part 4의 10번 · 11번
<code>lineage</code> 가 state의 고유 식별자 역할을 한다. 잘못된 state를 덮어쓰는 사고를 어떻게 막나	Part 4의 12번

오늘 다룰 것

참고

세션은 60분이고 그중 45분이 실습입니다. 그래서 이 문서는 대부분 연습용입니다. 라이브에서 다루는 것은 Part 2 · 3 · 6뿐입니다. 나머지는 읽고 오세요. 읽지 않으면 실습 중에 왜 이렇게 하는지 모른 채 명령만 따라 입력하게 됩니다.

Part	섹션	무엇을 다루나	언제
Part 0	용어 사전	오늘 처음 나오는 말 18개	예습
Part 1	1 · 2	로컬 state가 막히는 지점	예습

Part	섹션	무엇을 다루나	언제
Part 2	3 · 4 · 5	원격 backend와 닭-달걀 문제	라이브 4분
Part 3	6 · 7 · 8 · 9	잠금이 걸리는 순간	라이브 3분
Part 4	10 · 11 · 12 · 13	state 저장소를 안전하게 두기	예습
Part 5	14 · 15 · 16	오늘 다시 쓰는 week2 코드 읽기	예습
Part 6	17 · 18	비용과 정리	라이브 2분
Part 7	19	다음 주로 이어지는 문제	예습

오늘 나와야 하는 숫자

실습 중에 이 숫자와 다르면 멈추고 물어보세요.

지점	나와야 하는 값
bootstrap plan	Plan: 5 to add, 0 to change, 0 to destroy.
bootstrap state list	5줄
app plan (로컬 state)	Plan: 7 to add, 0 to change, 0 to destroy.
app state list	9줄 (리소스 7 + 데이터 소스 2)
app plan (원격 이전 직후)	No changes.
S3 객체	week03/app/terraform.tfstate 1개
DynamoDB 항목	apply 중에 하나 늘고, 끝나면 다시 줄어듭니다
app destroy	Destroy complete! Resources: 7 destroyed.

5 와 7 과 9 가 각각 다른 뜻입니다. 5는 bootstrap이 만드는 리소스, 7은 app이 만드는 리소스, 9는 app의 state list 줄 수입니다. state list 는 데이터 소스도 출력하기 때문에 9줄이 됩니다.

Part 0. 용어 사전

오늘 처음 나오는 단어를 실습 전에 모두 설명해 둡니다. week2에서 정의한 용어는 넣지 않았습니다. 비유는 이 표 안에서만 쓰고, 본문에서는 왼쪽 옆의 실제 이름으로 부릅니다.

용어	뜻	비유
backend	Terraform이 state를 어디에 두고 어떻게 잠글지 정하는 설정. 기본값이 local 이라 지금까지 terraform.tfstate 가 폴더에 생겼습니다	state의 주소

용어	뜻	비유
원격 state (remote state)	state 파일을 내 디스크 대신 공유 저장소에 두는 것	장부를 공용 금고에 둔다
state 잠금 (locking)	한 사람이 <code>apply</code> 하는 동안 다른 사람이 같은 state를 건드리지 못하게 막는 것	문서 편집기의 "다른 사용자가 사용 중"
LockID	잠금을 표시하는 항목의 키 이름. Terraform이 이 이름으로 찾습니다	-
S3 (Simple Storage Service)	AWS의 파일 저장소. 폴더 구조처럼 보이지만 실제로는 키와 값의 쌍입니다. <code>week03/app/terraform.tfstate</code> 전체가 키 하나입니다	-
버킷 (bucket)	S3에서 파일을 담는 최상위 단위. 이름이 전 세계에서 유일해야 합니다	-
버전 관리 (versioning)	같은 키에 새 파일을 덮어써도 이전 내용을 따로 보관하는 기능	실행 취소
SSE-S3 / AES256	서버측 암호화. S3가 관리하는 키로 저장할 때 암호화하며 추가 비용이 없습니다	-
Public Access Block	버킷이 실수로 공개되는 것을 막는 4가지 차단 설정	-
DynamoDB	AWS의 키-값 데이터베이스. 오늘은 데이터를 담으려고 쓰는 게 아니라, "지금 누가 작업 중"이라는 표시 항목 하나를 넣고 빼는 용도로만 씁니다	자물쇠를 걸어두는 못
hash_key (partition key)	DynamoDB에서 항목을 찾는 기준이 되는 키. 오늘은 <code>LockID</code> 로 고정합니다	-
PAY_PER_REQUEST / PROVISIONED	DynamoDB 요금제 두 가지. 앞은 요청한 만큼만 내고, 뒤는 처리 용량을 미리 잡아두고 쓰지 않아도 그만큼 냅니다	-
<code>-migrate-state</code>	<code>terraform init</code> 에 붙이는 옵션. backend가 바뀐 것을 감지하고 기존 state를 새 backend로 옮겨줍니다	이사
partial configuration	backend 블록의 값을 일부만 코드에 적고 나머지는 <code>-backend-config</code> 로 넘기는 방식	-
<code>use_lockfile</code>	Terraform 1.10부터 S3 backend가 지원하는 잠금 방식. DynamoDB 없이 S3에 잠금 파일을 만들어 잠급니다	-
<code>force-unlock</code>	비정상 종료로 잠금이 남았을 때 강제로 푸는 명령	자물쇠를 부순다
IMDSv2 (Instance Metadata Service v2)	EC2가 자기 정보를 조회하는 서비스의 2세대 방식. 토큰을 먼저 받아야 조회할 수 있어서, 서버 안까지 들어온 공격자가 자격증명을 쉽게 탈취하지 못합니다	-
<code>default_tags</code>	provider 블록에 한 번 적으면 그 스택이 만드는 모든 리소스에 자동으로 붙는 태그	-

Part 1. 로컬 state가 막히는 지점

[개념] 1. state 파일 안에 무엇이 들어 있었나

week2 실습워크북 B-4에서 직접 열어봤습니다. 최상위 키가 7개였고, `resources` 배열에 리소스마다 `attributes` 와 `dependencies` 가 들어 있었습니다.

정리하면 state는 세 가지를 담고 있습니다. 지금 관리 중인 리소스의 실제 ID와 속성값, 리소스 사이의 의존 관계, 그리고 이 state 자체의 이력인 `serial` 과 `lineage` 입니다.

그래서 state를 잃으면 Terraform은 자기가 무엇을 만들었는지 모릅니다. 코드는 그대로 있어도 "이미 있는 것"과 "새로 만들 것"을 구분할 수 없어서, `apply` 하면 같은 리소스를 또 만들려고 합니다.

[개념] 2. 로컬 state의 네 가지 한계

한계	무엇이 문제인가
공유	팀원이 내 state를 볼 수 없습니다. 각자 자기 state를 만들면 같은 인프라를 두 번 만듭니다
잠금	두 사람이 동시에 <code>apply</code> 하면 나중에 저장한 쪽이 앞서 사람의 기록을 덮어씁니다
유실	노트북을 포맷하면 state를 복구할 수 없습니다. 리소스는 AWS에 남아 있는데 지울 수단이 없어집니다
평문 시크릿	state는 암호화되지 않은 JSON입니다. DB 비밀번호나 개인키를 만들면 그대로 적힙니다

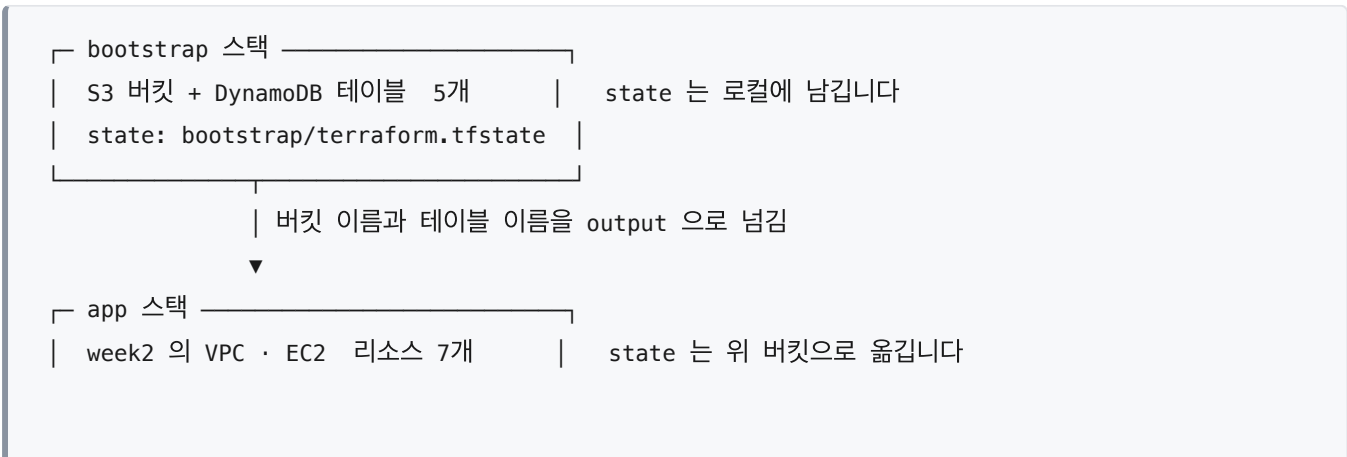
오늘 앞의 셋을 해결합니다. 네 번째는 완전히 해결되지 않습니다. 원격에 두면 암호화와 접근 제어를 걸 수 있지만 state 안의 값 자체는 여전히 평문입니다. 그래서 오늘 키페어를 만들지 않습니다(Part 4의 10번).

Part 2. 원격 backend와 닭-달걀 문제

[개념] 3. 닭-달걀 문제와 2-스택 패턴

state를 담은 S3 버킷과 DynamoDB 테이블도 Terraform으로 만들고 싶습니다. 그런데 그 리소스를 만드는 `apply` 의 state는 어디에 둘까요. 저장소가 아직 없습니다.

이것이 닭-달걀 문제입니다. 해결은 스택을 두 개로 나누는 것입니다.



```
state: s3://...-tfstate/week03/...
```

bootstrap의 state만 로컬에 남습니다. 자기가 만든 버킷에 자기 state를 넣으려면 그 버킷이 이미 있어야 하는데, 지금 그것을 만드는 중이기 때문입니다.

체크포인트

bootstrap 스택은 오늘 destroy하지 않습니다. app 스택의 state가 그 버킷 안에 들어 있습니다. 7주차에 지웁니다.

비용·보안 경고

`practice/bootstrap/terraform.tfstate` 를 지우지 마세요. `.gitignore` 대상이라 커밋되지 않고, 7주차에 버킷과 테이블을 지울 때 필요한 유일한 기록입니다. 잃으면 콘솔에서 직접 지워야 합니다.

[함정] 4. backend 블록에 변수를 쓸 수 없는 이유

예습 체크로 냈던 질문입니다. 답은 순서에 있습니다.

`terraform init` 은 backend를 가장 먼저 초기화합니다. 그 시점에는 아직 변수가 평가되지 않았습니다.

`terraform.tfvars` 를 읽는 것도, `var.project_name` 에 값을 채우는 것도 그 뒤의 일입니다. 그래서 backend 블록 안에는 `var.`, `local.`, 다른 리소스 참조를 쓸 수 없습니다.

```
terraform {
  backend "s3" {
    bucket = "${var.project_name}-tfstate" # 이렇게 쓸 수 없습니다
  }
}
```

그래서 오늘 버킷 이름을 직접 적습니다. 직접 적으면 오타가 생기기 쉬우므로, bootstrap에서 붙여넣을 다섯 줄을 그대로 출력하게 만들어 뒀습니다. 실습 A-4의 TODO ⑧입니다.

값을 코드 밖으로 빼는 방법이 아예 없는 것은 아닙니다. `-backend-config` 파일로 넘기는 partial configuration이 있고, 심화 과제 B-8에서 해봅니다.

[개념] 5. `terraform init -migrate-state` 가 하는 일

backend를 바꾸고 `init` 하면 Terraform은 이렇게 움직입니다.

- ① 이전 backend 인 local 에 state 가 있는지 확인합니다
- ② 있으면 새 backend 로 복사할지 묻습니다
- ③ yes 면 state 를 새 backend 에 쓰고, 로컬 파일은 백업으로 남깁니다

그래서 순서가 중요합니다. 먼저 로컬 state로 `apply` 를 끝내야 옮길 state가 생깁니다. backend 블록을 처음부터 켜두고 시작하면 옮길 것이 없어서 이 과정을 볼 수 없습니다. `practice/app/backend.tf` 가 주석 상태로 시작하는 이유입니다.

Part 3. 잠금이 걸리는 순간

[개념] 6. DynamoDB 잠금의 원리: 항목 하나

생각보다 단순합니다. Terraform은 `apply` 를 시작할 때 `LockID` 를 키로 항목 하나를 테이블에 넣고, 끝나면 지웁니다. 항목이 이미 있으면 넣기가 실패하고, 그 실패가 "다른 사람이 작업 중"이라는 신호가 됩니다.

`hash_key` 이름이 반드시 `LockID` 여야 하는 것도 이 때문입니다. Terraform이 그 이름으로 찾습니다.

테이블에는 잠금 항목 말고 state 체크섬 항목도 들어갑니다. 키가 `<버킷>/<키>-md5` 이고, state를 쓸 때마다 갱신되며 작업이 끝나도 남습니다. 그래서 B-7에서는 항목의 절대 개수보다, `apply` 중에 하나 늘고 끝나면 다시 줄어드는 움직임을 보세요.

실습 B-7에서 터미널 두 개로 재현하면 뒤늦게 들어온 쪽이 이런 메시지를 받습니다.

```
Error: Error acquiring the state lock
```

이때 함께 출력되는 `Lock Info` 블록에 누가 언제 잠갔는지가 적혀 있습니다. `Who` 항목에 사용자 이름과 호스트명이 들어가서, 팀 작업에서는 이것만 보고 연락할 수 있습니다.

[개념] 7. PAY_PER_REQUEST를 쓰는 이유

비용·보안 경고

`billing_mode` 를 `PROVISIONED` 로 두면 처리 용량을 미리 잡아두고 쓰지 않아도 그만큼 과금됩니다. 오늘 만드는 테이블은 실습 중에 항목 몇십 개를 넣고 빼는 정도라, `PAY_PER_REQUEST` 로 두면 사실상 \$0입니다.

숫자로 보면 서울 리전 온디맨드는 쓰기 100만 건에 \$0.68, 읽기 100만 건에 \$0.1355입니다. 실습 규모에서는 무시해도 될 수준입니다.

한 가지 알아둘 점이 있습니다. DynamoDB의 상시 프리티어는 스토리지 25GB와 프로비저닝된 용량 25 WCU/RCU입니다. 온디맨드 요청은 프리티어 대상이 아닙니다. 그러니 "프리티어라서 무료"가 아니라 "단가가 낮아서 실습 규모에서는 0에 가깝다"가 맞는 설명입니다.

[함정] 8. 잠금이 남았을 때: `force-unlock`

`apply` 중에 터미널을 강제로 닫거나 네트워크가 끊기면 잠금 항목이 지워지지 않고 남습니다. 그러면 다음 `plan` 부터 계속 잠금 오류가 납니다.

```
terraform force-unlock <LOCK_ID>
```

주의

이 명령은 아무도 작업하지 않는다는 것이 확실할 때만 씁니다. 실제로 다른 사람이 `apply` 중인데 강제로 풀면, 두 `apply` 가 같은 state를 동시에 쓰면서 state가 손상됩니다. 팀에서는 먼저 `Lock Info` 의 `Who` 를 보고 그 사람에게

확인하세요.

[개념] 9. `use_lockfile` 과의 비교 (심화 ★)

참고

Terraform 1.10부터 S3 backend가 `use_lockfile = true` 를 지원합니다. DynamoDB 없이 S3에 잠금 파일을 만들어 잠그는 방식입니다. 테이블을 따로 만들 필요가 없어서 요즘은 이쪽이 기본 선택입니다.

오늘 DynamoDB 방식을 쓰는 이유는 잠금이 걸리는 순간을 콘솔에서 눈으로 볼 수 있기 때문입니다. 테이블 항목이 생겼다 사라지는 것을 보면 6번의 설명이 그대로 확인됩니다.

그래서 `init` 할 때 이런 경고가 뜹니다. 정상입니다.

```
| Warning: Deprecated Parameter
|
|   on backend.tf line 7, in terraform:
|     7:     dynamodb_table = "boaz26-w3-kdh1834-tflock"
|
| The parameter "dynamodb_table" is deprecated. Use parameter "use_lockfile"
| instead.
```

line 7 과 7: 은 예시입니다. 경고는 `dynamodb_table` 이 실제로 놓인 줄을 가리키므로, `practice/app/backend.tf` 의 주석을 푼 상태에서는 20행으로 나옵니다.

`use_lockfile` 을 직접 써보는 것은 실습 B-9입니다. Terraform 1.10 이상이 필요합니다.

backend가 요구하는 IAM 권한

관리자 권한으로 실습하면 신경 쓸 일이 없지만, 회사에서 쓸 때는 이 목록이 최소 권한입니다.

대상	필요한 동작
state 버킷	<code>s3:ListBucket</code>
state 객체	<code>s3:GetObject</code> · <code>s3:PutObject</code> · <code>s3:DeleteObject</code>
잠금 테이블	<code>dynamodb:DescribeTable</code> · <code>dynamodb:GetItem</code> · <code>dynamodb:PutItem</code> · <code>dynamodb:DeleteItem</code>

권한이 없으면 `init` 이나 `plan` 에서 `AccessDenied` 가 발생합니다. 실습 B-5의 함정 항목입니다.

Part 4. state 저장소를 안전하게 두기

bootstrap 스택이 리소스를 5개나 만드는 이유가 여기 있습니다. 버킷 하나만 만들어도 state는 저장됩니다. 나머지 넷은 안전을 위한 것입니다.

[개념] 10. state는 평문 JSON입니다

state 안에는 시크릿이 섞여 들어갑니다. RDS를 만들면 초기 비밀번호가, `tls_private_key` 로 키를 만들면 개인키가 그대로 적힙니다.

그래서 두 가지를 합니다. 하나는 저장할 때 암호화하는 것입니다. `sse_algorithm = "AES256"` 이 그것이고, S3가 관리하는 키를 쓰는 방식이라 추가 비용이 없습니다.

다른 하나는 애초에 시크릿을 만들지 않는 것입니다. 오늘 키페어를 만들지 않는 이유가 이것입니다. `tls_private_key` 로 키를 만들면 개인키가 state에 평문으로 남고, `.pem` 파일이 실습 폴더에 남아 PR로 커밋되는 사고가 발생합니다.

그래서 오늘 성공 기준은 SSH 접속이 아닙니다. week2와 같이 `instance_state = running` 과 퍼블릭 IP 할당까지입니다.

[개념] 11. 퍼블릭 접근 차단 4종

state 버킷이 공개되면 인프라 전체 구성 정보가 외부에 노출됩니다. VPC CIDR, 서브넷 구성, 시큐리티 그룹 규칙, 인스턴스 ID가 다 적혀 있습니다.

`aws_s3_bucket_public_access_block` 은 스위치 4개를 모두 `true` 로 둡니다. 나중에 실수로 공개 정책을 붙이려 해도 이 리소스가 먼저 거부합니다.

[개념] 12. 버전 관리: 같은 키에 덮어써도 이전 버전이 남습니다

state 저장소에서는 반드시 켜야 합니다.

S3는 같은 키에 새 파일을 올리면 기본적으로 이전 내용을 버립니다. 버전 관리를 켜면 이전 내용을 별도 버전으로 보관합니다. `week03/app/terraform.tfstate` 라는 키는 하나지만, 그 아래에 apply할 때마다 버전이 쌓입니다.

이것이 잘못된 state를 덮어썼을 때 되돌릴 수 있는 유일한 수단입니다. week2에서 `lineage` 가 state의 고유 식별자라고 했는데, lineage가 다른 state를 덮어썼을 때 이전 버전으로 복원할 수 있게 해주는 것이 버전 관리입니다.

[함정] 13. `force_destroy` 를 켜지 않는 이유

주의

`aws_s3_bucket` 에 `force_destroy = true` 를 넣으면 `terraform destroy` 한 번에 버킷 안의 모든 객체와 모든 버전이 삭제됩니다. 12번에서 만든 안전장치가 무의미해집니다.

그래서 코드에 넣지 않았습니다. 대신 버킷을 지우는 절차를 실습워크북 C-5에 따로 적어 뒀습니다. 버전 관리가 켜진 버킷은 모든 버전과 삭제 마커를 먼저 비워야 `DeleteBucket` 이 통과합니다.

Part 5. 오늘 다시 쓰는 week2 코드 읽기

app 스택의 `network.tf` 와 `compute.tf` 는 week2 정답 코드와 같습니다. 오늘은 채우지 않고 읽기만 합니다. 다만 그 안에 week2에서 다룬 주의 사항 설명이 들어 있으니 한 번 읽어주세요.

[개념] 14. 참조 ①~⑩ 의존성 지도

번호는 의존성 층 순서입니다. 파일에 적힌 줄 순서와는 다릅니다. week2 실습워크북의 지도와 같은 번호입니다.

```
0층 · 먼저 AWS에서 조회하는 것
    data.aws_ami.al2023                data.aws_availability_zones.available

1층 · 아무도 안 기다리는 유일한 리소스
    aws_vpc.main

2층 · VPC만 있으면 되는 셋
    aws_subnet.public      <- ① vpc      <- ② AZ데이터
    aws_internet_gateway.gw <- ③ vpc
    aws_security_group.web  <- ⑧ vpc

3층 · 2층이 끝나야 시작
    aws_route_table.public    <- ④ vpc      <- ⑤ igw
    aws_instance.web          <- ⑨ subnet    <- ⑩ sg      <- ⑪ ami데이터

4층 · 맨 마지막
    aws_route_table_association.public <- ⑥ subnet <- ⑦ route_table
```

<- 는 "이 값을 기다린다"는 뜻입니다. ②와 ⑩이 오늘 나오는 두 개의 `data` 참조입니다.

라우트 테이블을 서브넷에 붙이는 `aws_route_table_association` 은 인자가 참조 두 줄뿐인데 가장 마지막입니다. 연결 자체가 하나의 리소스이고, 이게 없으면 IGW가 있어도 밖으로 나갈 수 없습니다.

[함정] 15. AMI 필터를 엄격하게 지정하는 이유

`compute.tf` 에서 이름 패턴을 `al2023-ami-2023.*` 로 시작까지 고정합니다. `al2023-ami-*-x86_64` 처럼 느슨하게 두면 아래 것들이 후보에 들어오고 `most_recent = true` 가 그중 최신을 선택합니다.

조건에 걸리는 AMI	무엇이 문제인가
<code>al2023-ami-ecs-neuron-hvm-*</code>	ECS 전용. 루트 볼륨 스냅샷이 30GiB라서 <code>volume_size = 8</code> 과 충돌해 apply가 실패합니다
<code>al2023-ami-ecs-hvm-*</code>	ECS 전용
<code>al2023-ami-minimal-*</code>	SSM Agent·awscli 등이 빠진 최소 이미지

`owners = ["amazon"]` 은 이걸 막아주지 못합니다. ECS AMI를 발행하는 AWS 서비스팀 계정도 소유자 별칭이 `amazon` 이기 때문입니다.

[개념] 16. 그 밖에 코드에 적어둔 것들

코드	뜻
<code>map_public_ip_on_launch = true</code>	이 서버넷에서 뜨는 인스턴스가 퍼블릭 IPv4를 자동으로 받습니다. EIP를 만들지 않아도 인터넷에서 닿는 주소가 붙는 이유가 이 한 줄입니다
<code>associate_public_ip_address</code> 를 안 쓰는 것	퍼블릭 IP는 서버넷 옵션 하나로만 통제합니다. 두 곳에 적으면 설정이 나뉘고, 나중에 값을 바꿀 때 인스턴스가 통째로 재생성 (-/+)됩니다
<code>metadata_options { http_tokens = "required" }</code>	IMDSv2 강제. AL2023 기본값이지만 코드에 의도를 남깁니다
인라인 <code>ingress / egress</code>	별도 리소스인 <code>aws_vpc_security_group_ingress_rule</code> 과 섞어 쓰면 서로의 규칙을 지웁니다. 오늘 코드는 인라인 방식만 씁니다
<code>data.aws_availability_zones</code>	AZ 이름은 계정마다 물리 AZ에 다르게 매핑됩니다. 그래서 하드코딩하지 않고 AWS에서 조회합니다

Part 6. 비용과 정리

[개념] 17. 이번 주 비용

app 스택을 apply하면 서버 한 대가 켜집니다. 켜져 있는 동안 요금이 발생합니다.

항목	시간당	730시간(한 달)
EC2 <code>t3.micro</code> (서울)	\$0.0130	\$9.49
퍼블릭 IPv4 주소 1개	\$0.0050	\$3.65
EBS <code>gp3</code> 루트 8GiB	\$0.0010	\$0.73
VPC · 서버넷 · IGW · 라우트 테이블 · 연결 · 시큐리티 그룹	\$0	\$0
app 스택 합계	\$0.0190	\$13.87

bootstrap 스택은 따로 계산합니다. 7주차까지 남겨두기 때문입니다.

항목	한 달
S3 Standard (state 수십 KB)	\$0.01 미만
S3 요청 (PUT/LIST · GET)	\$0.01 미만
DynamoDB 온디맨드 요청	\$0.01 미만

항목	한 달
DynamoDB 스토리지 (25GB까지 무료)	\$0
bootstrap 스택 합계	\$0.01 미만

상황	총 비용
3시간 실습하고 바로 destroy	약 \$0.06
destroy를 잊고 한 달 방치	약 \$13.87

차이는 약 243배이고, 그 차이를 만드는 것은 `terraform destroy` 한 줄입니다.

비용·보안 경고

"프리티어니까 괜찮겠지"로 넘기지 마세요. 퍼블릭 IPv4는 2024-02-01부터 별도 과금이고, 2025-07-15 이후에 만든 계정은 프리티어 체계 자체가 다릅니다. 프리티어 여부와 무관하게 destroy를 안 하면 비용이 발생한다고 생각하고 진행하세요.

그리고 stop은 destroy가 아닙니다. 인스턴스를 멈추면 EC2 요금과 퍼블릭 IPv4 요금은 멈추지만, EBS 8GiB의 월 \$0.73은 계속 발생합니다.

[개념] 18. destroy 순서와 잔존 점검

순서가 정해져 있습니다. app을 먼저 지우고, bootstrap은 7주차까지 남깁니다. 반대로 하면 state 저장소가 먼저 사라져서 app 스택을 지울 수단이 없어집니다.

그리고 확인을 두 가지로 나눠서 합니다. state가 빈 것과 계정이 빈 것은 다른 확인입니다.

`terraform state list`가 비었다는 것은 Terraform이 더 이상 관리하지 않는다는 뜻일 뿐입니다. 콘솔에서 직접 만든 것이나 destroy가 중간에 실패해 남은 것은 여기에 나타나지 않습니다.

계정 쪽 확인에는 `default_tags`가 쓰입니다. `Purpose`가 두 값으로 나뉘어 있어서 지울 것과 남길 것을 태그로 구분할 수 있습니다.

Purpose	스택	오늘 세션 끝에
workload	app	0개여야 합니다
terraform-state	bootstrap	남아 있어야 정상입니다

`scripts/check-leftover.sh`가 이 구분대로 조회합니다. 실습 C-3에서 실행합니다.

Part 7. 다음 주로 이어지는 문제

[개념] 19. 세 문장으로 줄이면

1. state를 옮기는 것은 backend 블록 하나와 `init -migrate-state` 한 번입니다. 어려운 것은 옮기는 행위가 아니라, 옮길 저장소를 먼저 만들어야 한다는 순서입니다.
2. 잠금은 항목 하나입니다. apply 시작할 때 넣고 끝나면 지웁니다. 그 항목이 이미 있으면 남이 작업 중이라는 뜻입니다.
3. state 저장소는 버킷 하나로 끝나지 않습니다. 버전 관리 · 암호화 · 퍼블릭 차단 · 잠금 테이블까지 넷을 더 걸어야 state를 맡길 수 있습니다.

그리고 오늘 만든 것들이 다음 주의 문제를 그대로 만들어냅니다.

오늘 본 것	Week4에서 문제가 되는 지점
<code>backend.tf</code> 에 버킷 이름을 직접 적었습니다	backend는 변수를 못 받습니다. 그럼 나머지 값은 어디까지 변수로 분리할 수 있을까요 (<code>variable</code> 타입, <code>-backend-config</code>)
<code>project_name</code> 하나로 리소스 12개의 이름을 조립했습니다	접두어 하나로 되지 않고 이름 목록 자체가 여러 개라면 어떻게 쓸까요 (<code>for_each</code> vs <code>count</code>)
<code>"\${var.project_name}-tfstate"</code> 같은 조립을 여러 파일에 반복했습니다	조립식을 한 곳에 모아 관리할 방법이 필요합니다 (<code>locals</code>)
bootstrap의 output을 사람이 읽어 app에 옮겨 적었습니다	output 문법 자체를 다음 주에 본격적으로 다룹니다 (<code>output</code> , <code>sensitive</code>)
<code>t3.micro</code> · <code>10.0.1.0/24</code> · <code>AES256</code> 이 코드에 직접 적혀 있습니다	하드코딩된 값을 variables로 분리합니다

참고 문서

- [Backend block](#) : 4번의 출처
- [S3 backend](#) : 5·6·9번. `use_lockfile` 과 `dynamodb_table`
- [State Locking](#) : 6·8번
- [terraform init](#) : 5번의 `-migrate-state`
- [S3 버전 관리 \(AWS 문서\)](#) : 12번
- [S3 퍼블릭 액세스 차단 \(AWS 문서\)](#) : 11번
- week2 개념워크북 0·1·2·7·9·12번 : 오늘 다시 설명하지 않은 VPC · 참조 · 그래프 · AMI · state 구조 · 교체 규칙

이 문서의 비용은 서울 리전(ap-northeast-2) AWS 공개 가격 정보에서 조회한 값이고, deprecation 경고는 Terraform 1.15.8 · AWS provider 6.58.0 에서 실제로 재현한 값입니다. 원화 환산은 넣지 않았습니다. 환율은 조회하지 않았습니다.

다음 → 실습워크북 을 열고 practice/bootstrap/ 폴더로 이동하세요.